Unit 7 – AVL Tree

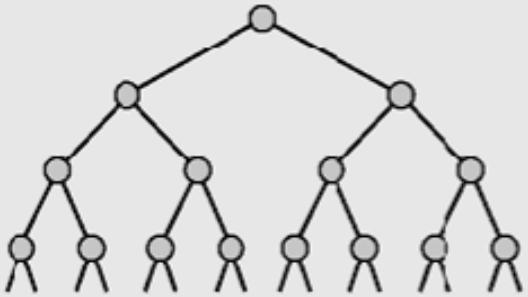
CS 201 - Data Structures II
Spring 2024
Habib University

Syeda Saleha Raza

Importance of height of BST

- Recall from the last session that the height of a binary search tree with n nodes can vary from log n (balanced) to n (degenerate).
- Height of a BST is critical because the complexity of find, insert, delete operations depend on the height of tree.
- A binary tree is height-balanced or balanced if the difference in height of both subtrees of any node in the tree is either zero or one

No. of nodes at different heights



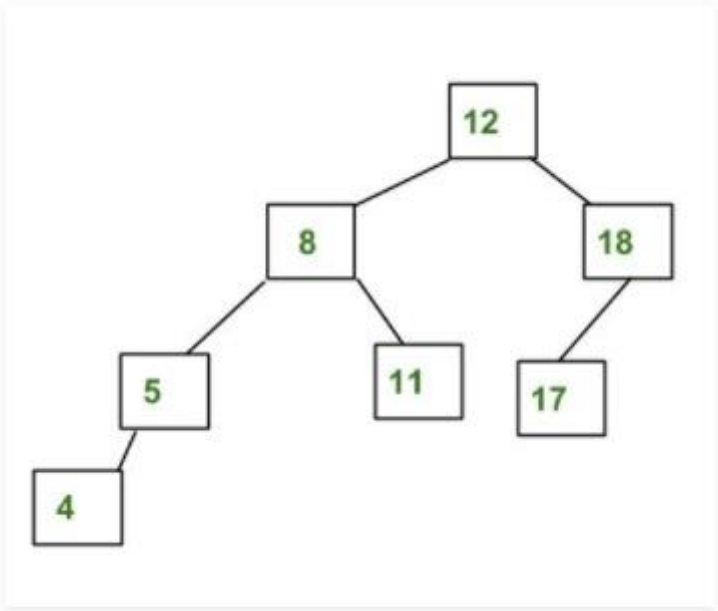
Height	*Nodes at One Level*	*Nodes at All Levels*
1	$2^0 = 1$	$1 = 2^1 - 1$
2	$2^1 = 2$	$3 = 2^2 - 1$
3	$2^2 = 4$	$7 = 2^3 - 1$
4	$2^3 = 8$	$15 = 2^4 - 1$
⋮		
11	$2^{10} = 1{,}024$	$2{,}047 = 2^{11} - 1$
⋮		
14	$2^{13} = 8{,}192$	$16{,}383 = 2^{14} - 1$
⋮		
h	2^{h-1}	$n = 2^h - 1$
⋮		

Self-balancing Tree

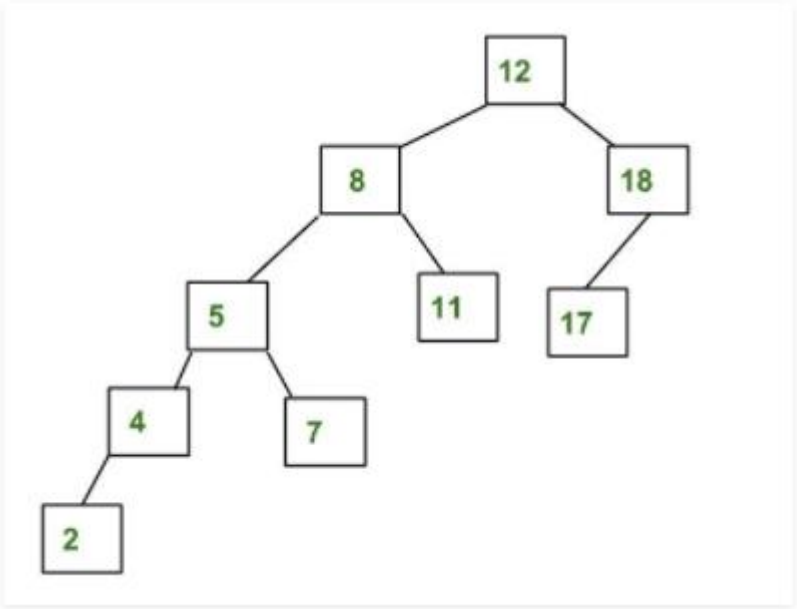
AVL Tree

- AVL tree is a self-balancing Binary Search Tree (BST) where the difference between heights of left and right subtrees cannot be more than one for all nodes.

Are these AVL trees?



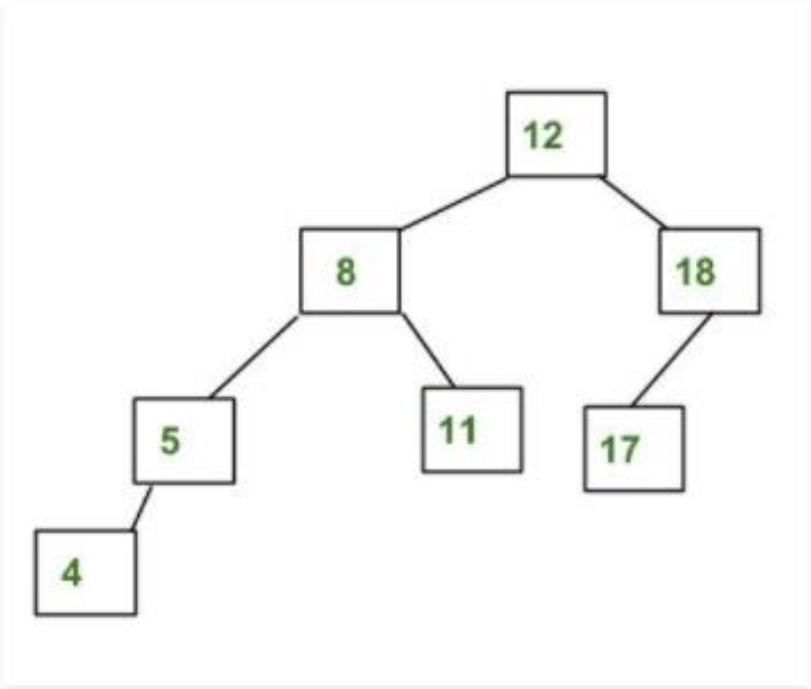
AVL Tree



Not an AVL Tree

Height-balance property

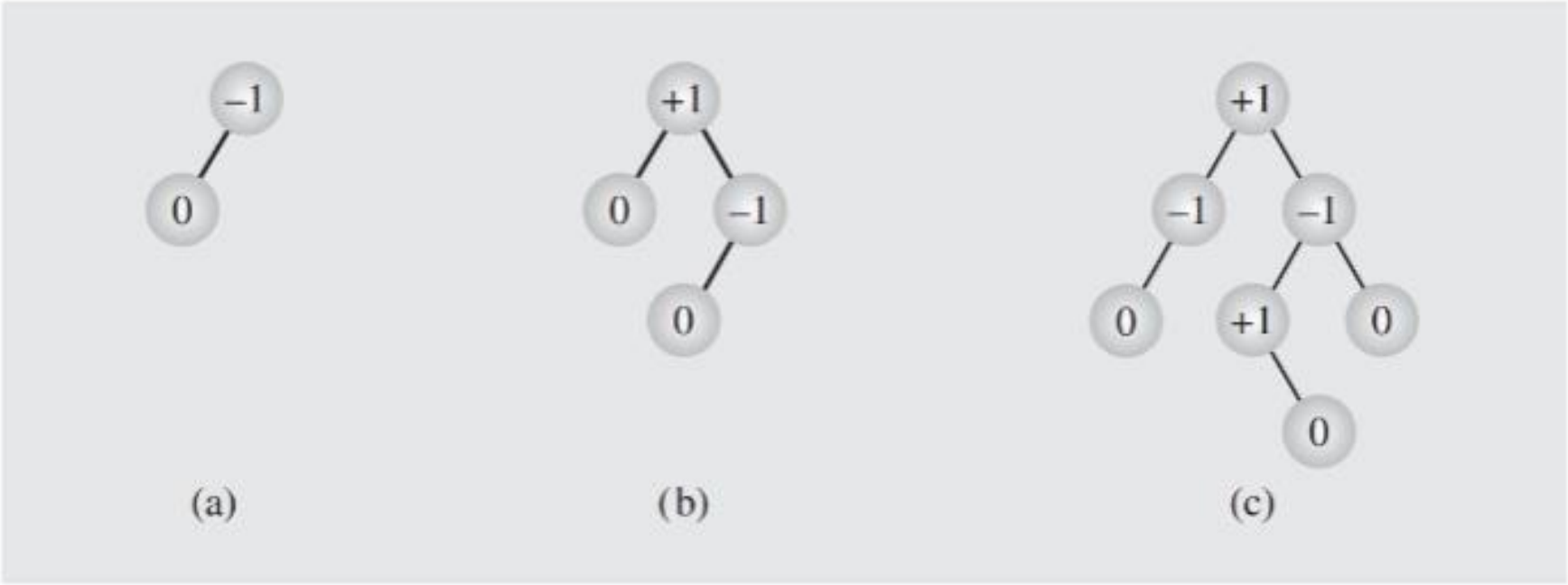
***Height-Balance Property*:** For every position p of T, the heights of the children of p differ by at most 1.



AVL Trees

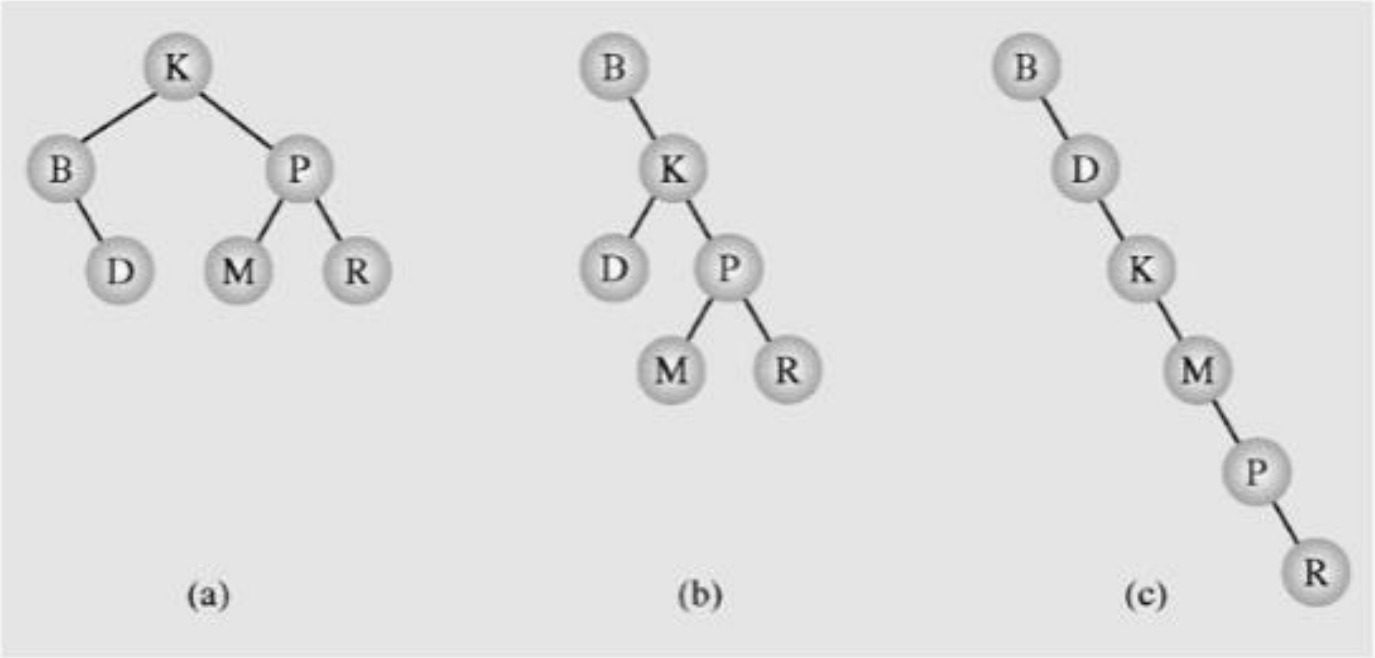
- We define balance factor as:
  – Balanced factor = $H_{right} - H_{left}$
- For an AVL tree, balance factors of every node should be +1, 0, or –1.
  - Every subtree in AVL Tree is also an AVL Tree
- If the balance factor of any node is >1 or <-1, we must balance it to make the balance factor between +1,0,-1
  - We do one or more rotation operations to do this balancing as we will see later
- An AVL tree is rebalanced after each insertion or deletion.
  - The height-balance property ensures that the height of an AVL tree with n nodes is O(log n).

Examples



Exercise

- Calculate balance factor of each node in the given BSTs. Are they AVL Tree?



Constructing an AVL Tree

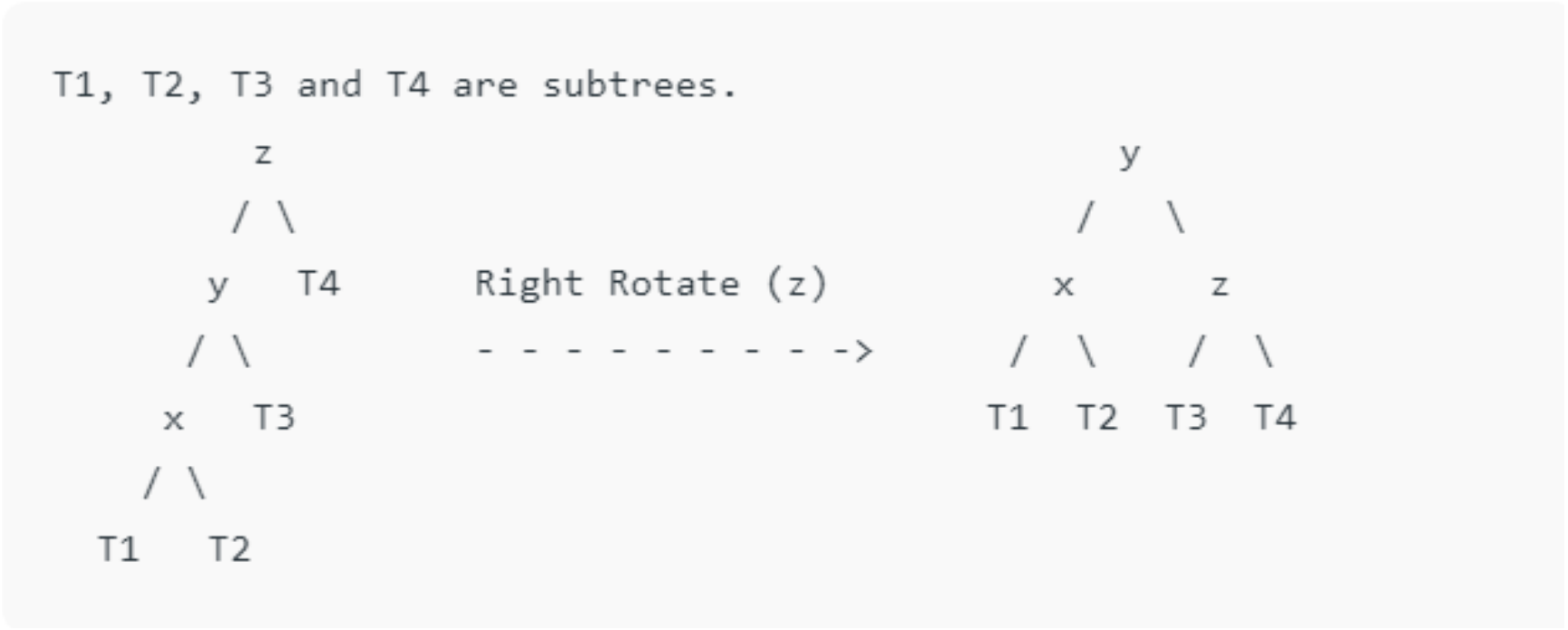
AVL - Insertion

- You want to inset a new node w in an AVL tree.
- Perform standard BST insert for w.
- Starting from w, travel up and find the first unbalanced node.
- Let z be the first unbalanced node, y be the child of z that comes on the path from w to z and x be the grandchild of z that comes on the path from w to z.
- Re-balance the tree by performing appropriate rotations on the subtree rooted with z. There can be 4 possible cases that need to be handled as x, y and z can be arranged in 4 ways.

Fixing height disbalance

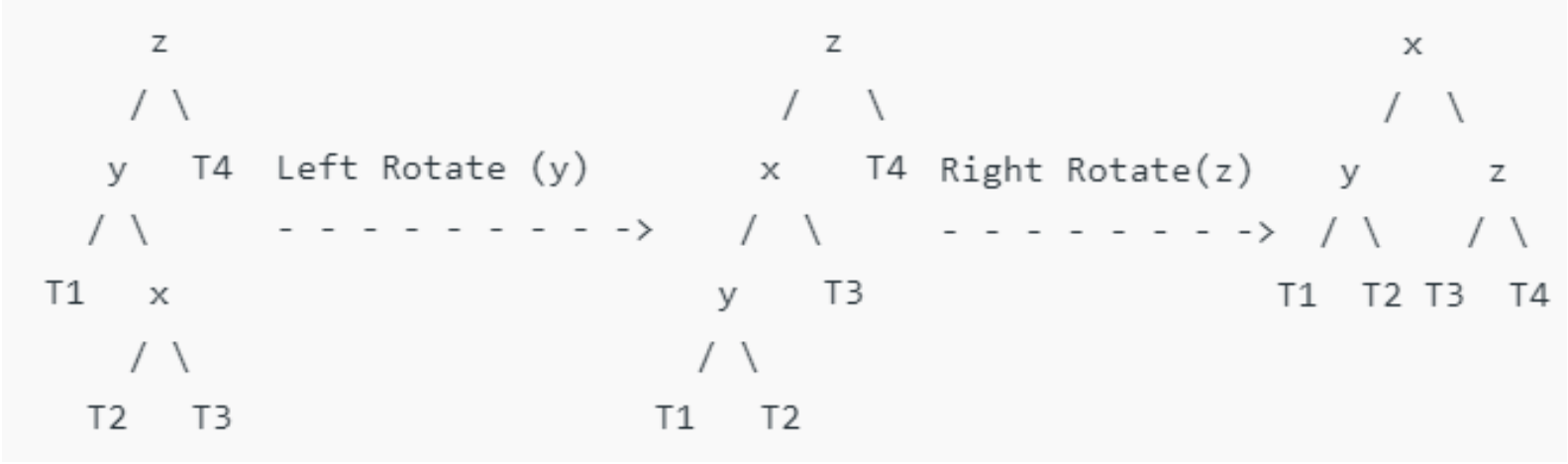
- In case of height disbalance, there are four possible case:
  - Left-Left
  - Left-Right
  - Right-Right
  - Right-Left

Left-Left Case



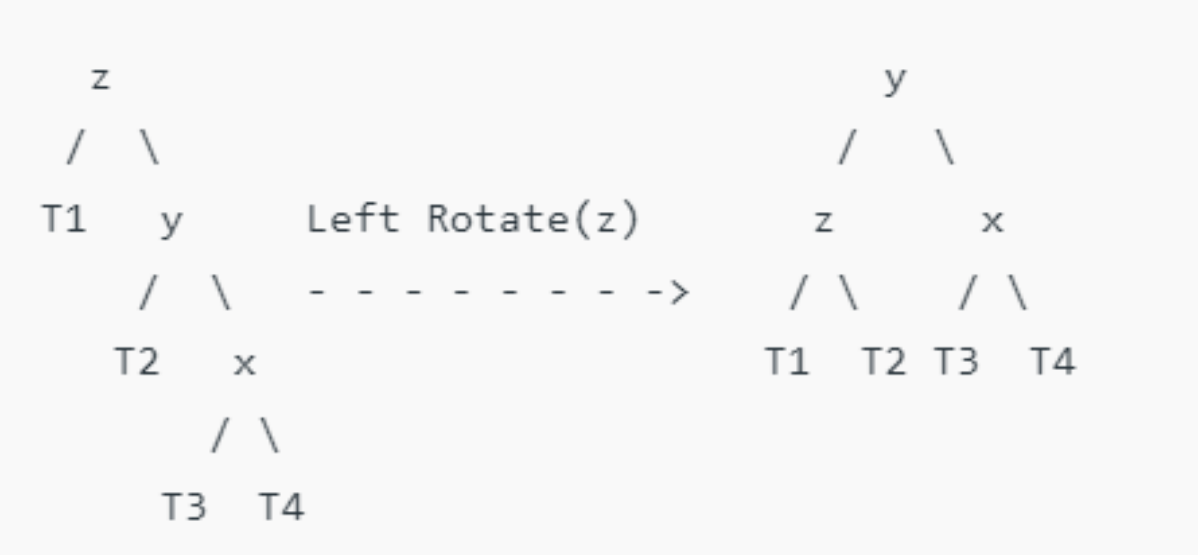
https://www.geeksforgeeks.org/insertion-in-an-avl-tree/?ref=lbp

Left-Right Case



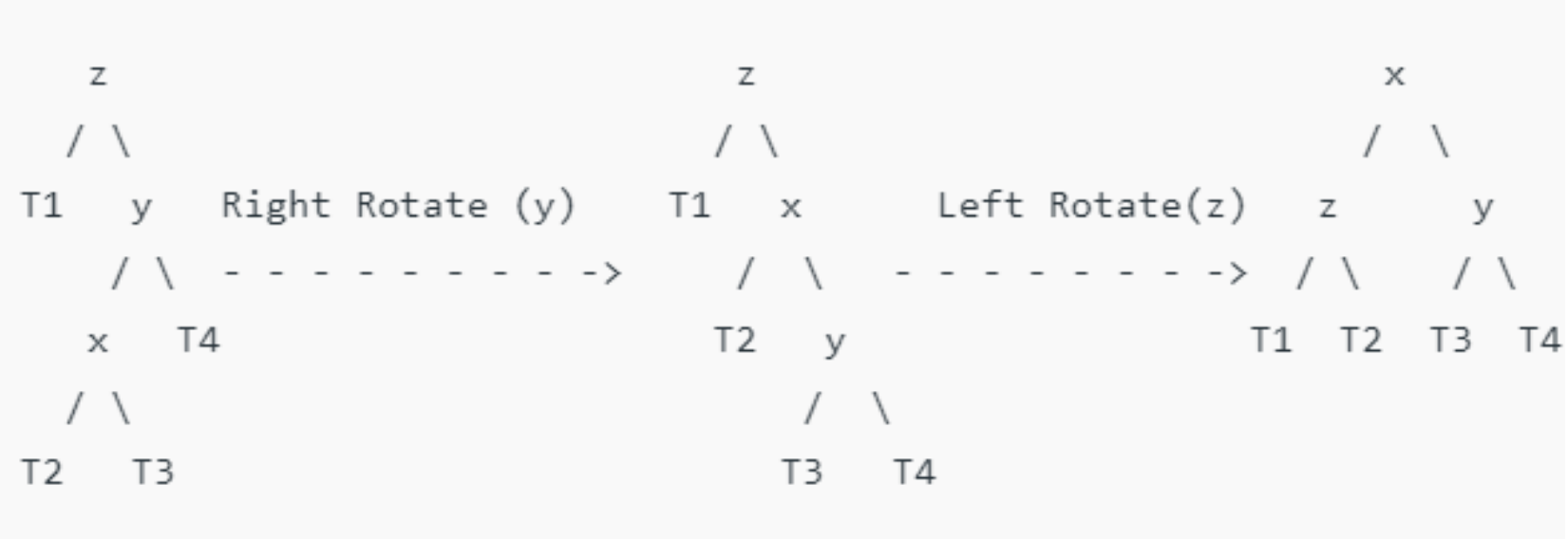
https://www.geeksforgeeks.org/insertion-in-an-avl-tree/?ref=lbp

Right-Right Case



https://www.geeksforgeeks.org/insertion-in-an-avl-tree/?ref=lbp

Right-Left Case



https://www.geeksforgeeks.org/insertion-in-an-avl-tree/?ref=lbp

Inserting a node in AVL Tree

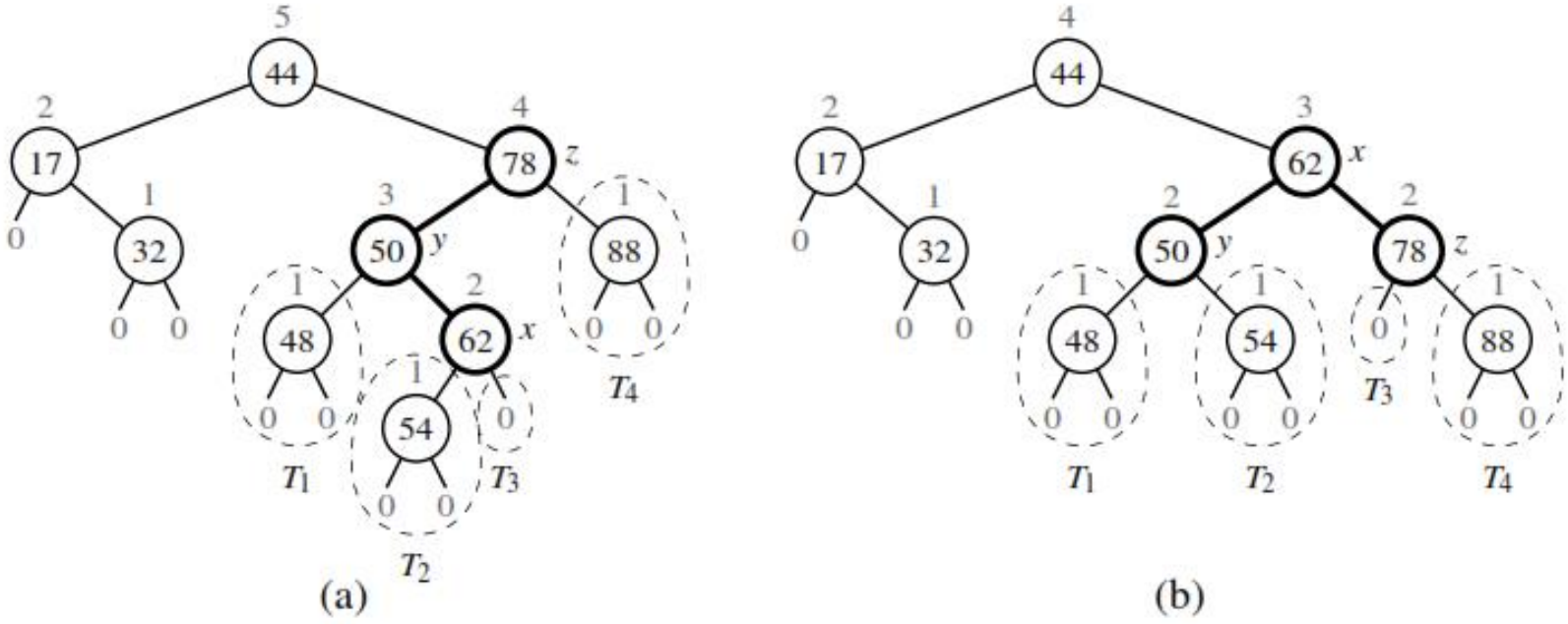


Figure 11.12: An example insertion of an item with key 54 in the AVL tree of Figure 11.11: (a) after adding a new node for key 54, the nodes storing keys 78 and 44 become unbalanced; (b) a trinode restructuring restores the height-balance property. We show the heights of nodes above them, and we identify the nodes x, y, and z and subtrees T_1, T_2, T_3, and T_4 participating in the trinode restructuring.

Example

- Let's construct an AVL tree with given values:
  - 55, 32, 9, 11, 48, 4, 20, 15

Exercise - Building a AVL tree

- 4,9,13,2,18,15,6,1

AVL Tree Visualisation

- Binary Search Tree, AVL Tree – VisuAlgo
- https://www.cs.usfca.edu/~galles/visualization/AVLtree.html

Deletion in AVL

- Let w be the node to be deleted
  - Perform standard BST delete for w.
  - Starting from w, travel up and find the first unbalanced node. Let z be the first unbalanced node, y be the larger height child of z, and x be the larger height child of y.
  - Re-balance the tree by performing appropriate rotations (discussed in AVL-Insertion) on the subtree rooted with z

Deleting a node in AVL Tree

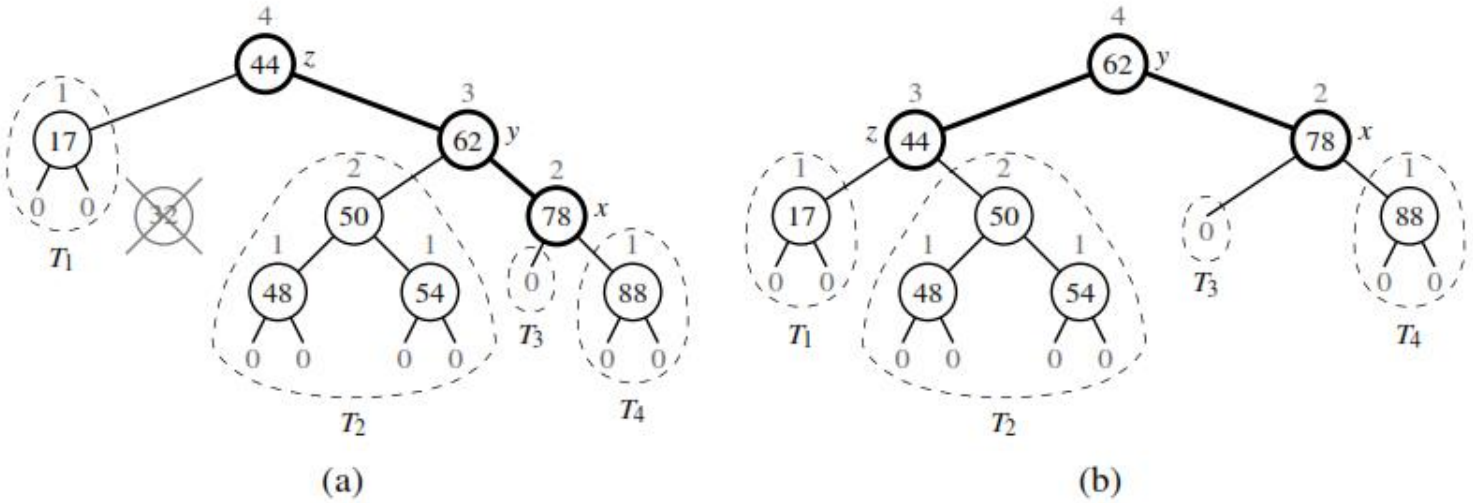
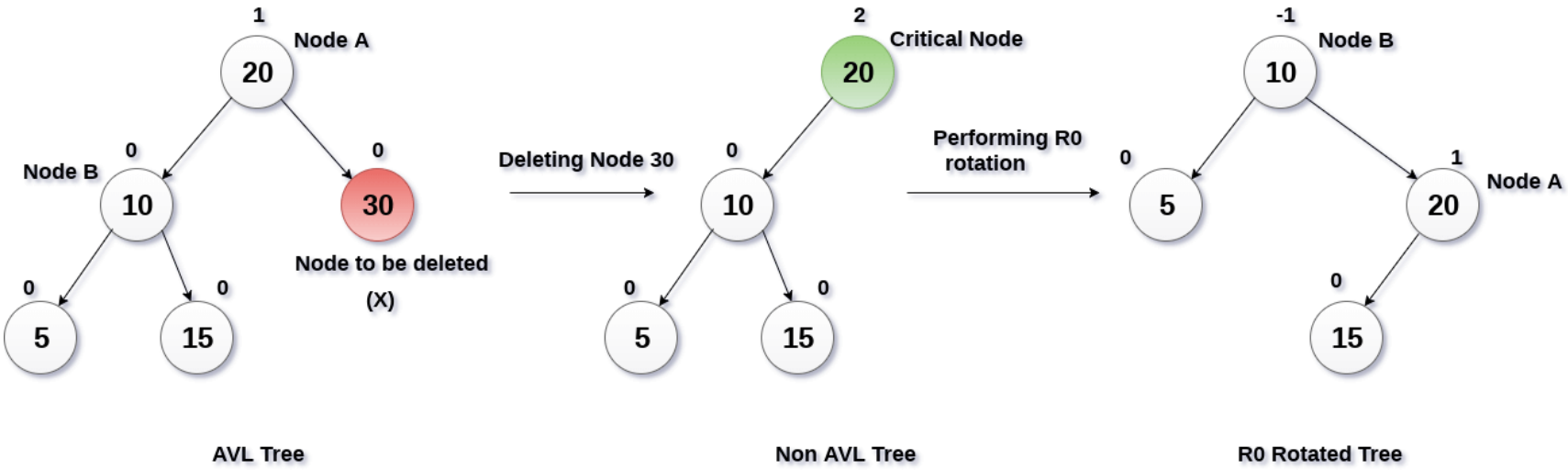


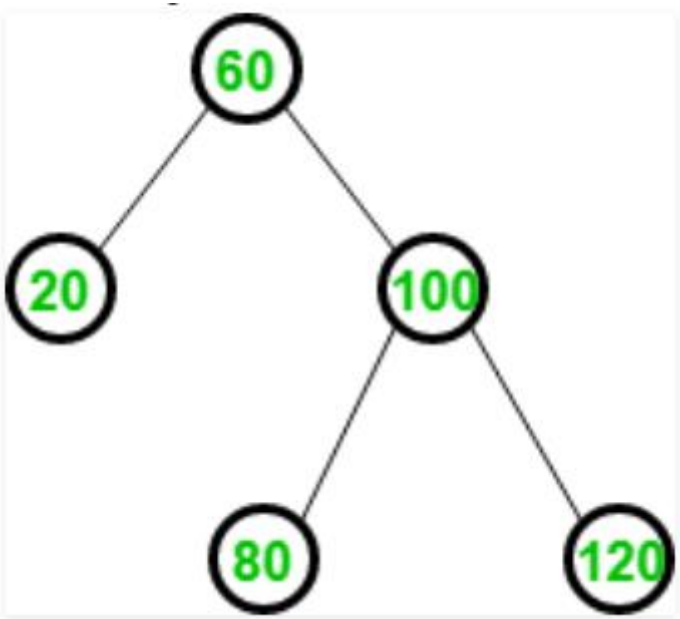
Figure 11.14: Deletion of the item with key 32 from the AVL tree of Figure 11.12b: (a) after removing the node storing key 32, the root becomes unbalanced; (b) a (single) rotation restores the height-balance property.

Deletion - Example



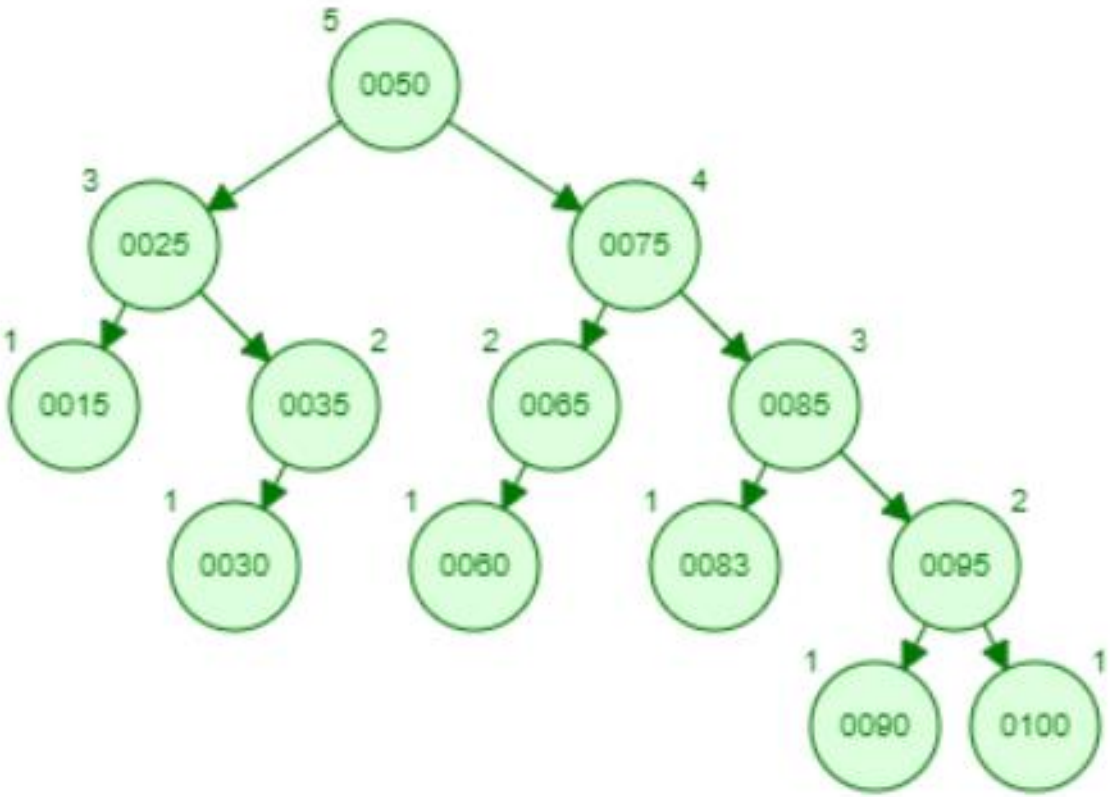
Exercise

- Remove 20 from this tree.



Exercise

- Delete 15 from the given tree.



Resources

- Open Data Structures (pseudocode edition), by Pat Morin. Available online at http://opendatastructures.org
- Data Structures and Algorithms in Python, by Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser. 2013. (1st. ed.). Wiley Publishing
- Insertion in an AVL Tree - GeeksforGeeks

Thanks